

Building an Agent Skill for Music Theory Analysis

Simran Mallik • Georgia Tech • Based on: *Science Skills for Antigravity* (Google DeepMind, 2026)

The Problem: Agents Without Guidance Are Unreliable

- According to the Google DeepMind paper, scientific agents without guidance consistently fail in predictable ways:

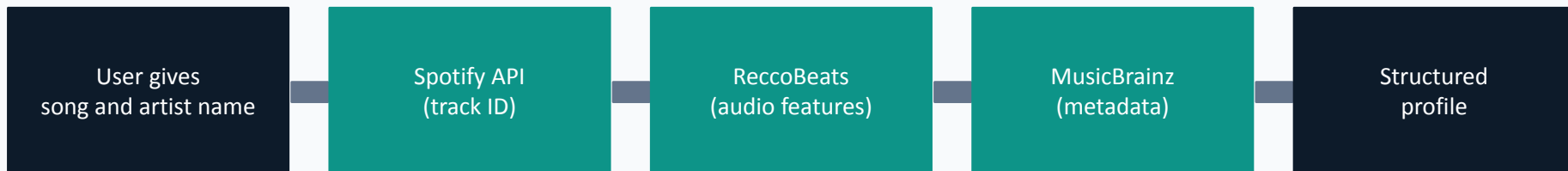
Hallucinated answers

Less efficient

Incorrect Interpretations

The Science Skills paper fix: give the agent a plain-text skill file with 3 components → reliability jumps from 49% to 93% and token usage drops by nearly half.

What I Built: Song Music Theory Analysis



Why music theory?

- ✓ Domain I know well (choir, a cappella, guitar, piano, music theory)
- ✓ Repeatable, multi-step workflow with 2+ external APIs
- ✓ Outputs are verifiable (I can check if the key and mood are correct)

Model used: llama-3.3-70b-versatile (Groq) • Framework: custom ReAct loop in Python

The Skill File: 3 Required Components

1

Behavioral Guidance

How to approach the task. Includes the mood synthesis table which converts valence + energy into a label (Euphoric/Upbeat, Angry/Intense, etc.) and defines what counts as high vs low.

2

Resource Pointers

Exact API endpoints, auth method, and response structure for Spotify, ReccoBeats, and MusicBrainz. Agent should never need to search for these.

3

Step-by-Step Workflow

6 numbered steps: authenticate → search Spotify for track ID → search ReccoBeats to get audio features → search MusicBrainz for release date and genre → convert key integer and synthesize mood → output structured profile.

Experiment Setup

BASELINE

System prompt:

"You are a helpful music analysis assistant."

Given 3 tools: Spotify, ReccoBeats, MusicBrainz. Agent figures out approach on its own.

WITH SKILL

System prompt:

Full skill file injected as context.

Agent has behavioral guidance, resource pointers, and workflow steps. Given same tools as baseline model.

3 Test Types (from the paper):

Unit Test

"What is the Spotify track ID for 'Shape of You' by Ed Sheeran?"

- One specific step in isolation

Workflow Test

"Give me a full music theory analysis of 'Blinding Lights' by The Weeknd"

- Full end-to-end
- All 3 APIs called, all fields interpreted?

Capability Test

"I'm building an upbeat playlist. Can you analyze 'Happy' by Pharrell Williams and tell me if it fits the mood?"

- Open-ended: agent applies skill without being told to?

Workflow Test Walkthrough — BASELINE (No Skill)

Prompt: "Give me a full music theory analysis of 'Blinding Lights' by The Weeknd"

Turn 1

`spotify_search("Blinding Lights by The Weeknd")`

✓ Got track ID: 0VjIjW4G1UZAMYd2vXMi3b

Turn 2

`reccobeats_features("0VjIjW4G1UZAMYd2vXMi3b")`

✓ key: C# major, tempo: 171, valence: 0.334, energy: 0.73

Turn 3

`musicbrainz_metadata("Blinding Lights")`

✗ release_date: Unknown, genres: Unknown

✓ **Final Output: PASS**

AI 3 APIs called. Audio features retrieved and interpreted correctly. Release date and genre unavailable from MusicBrainz, but this didn't affect the music theory analysis. No structured format or explicit mood label.

Workflow Test Walkthrough — WITH SKILL

Same prompt, same model, same tools. Skill injected as system prompt

Turn 1 `spotify_search("Blinding Lights by The Weeknd")`

✓ Got track ID: 0VjIjW4GluZAMYd2vXMi3b

Turn 2 `reccobeats_features("0VjIjW4GluZAMYd2vXMi3b")`

✓ key: C# major, tempo: 171, valence: 0.334, energy: 0.73

Turn 3 `musicbrainz_metadata("Blinding Lights")`

✗ release_date: Unknown, genres: Unknown

✓ **Final Output: PASS**

Exact structured format followed. All values interpreted (e.g. "0.334 (sad/tense)"). Release date and genre unavailable from MusicBrainz, but this didn't affect the music theory analysis. Correct mood label: Angry/Intense. Consistent 2-3 sentence narrative analysis.

Results: Reliability & Efficiency

Test	Type	Baseline Turns	Baseline Result	Skill Turns	Skill Result
1	Unit Test	1	Pass	3	Pass (Partial)
2	Workflow Test	3	Pass	3	Pass
3	Capability Test	2	Pass	3	Pass

3 / 3

Both conditions
passed all tests

6 vs 9

Total turns
(baseline vs skill)

0 vs 3

Tests with structured
format (base vs skill)

Did It Work? Comparing to the Paper

Metric	Paper Reports	Our Result	Match?
Reliability	49% → 93% with skill (+44pp)	Both conditions passed all 3 tests	~ Partial (baseline stronger than expected)
Efficiency (turns)	Skill reduces token usage	Skill used more turns (6 vs 9 total)	✗ Opposite (skill over-applied)
Consistency	Skill enforces repeatable workflow	Skill used structured format in 3/3 tests; baseline in 0/3	✓ Confirmed

The skill's main contribution was consistency and completeness, not correctness. The baseline was capable but unpredictable.

What Makes a Good Skill? What Was Hardest to Get Right

1

Behavioral Guidance

Medium

Straightforward to write once the domain was clear. The mood synthesis table was the key design decision. Binary thresholds (< 0.5 / ≥ 0.5) kept agent behavior consistent and testable.

2

Resource Pointers

Hardest

Hardest in practice, not because they were hard to write, but because API reliability was outside the skill's control. Spotify returned covers/remixes; ReccoBeats had missing songs. A skill can't compensate for bad upstream data.

3

Workflow Steps

Hard

The parallel tool call bug revealed that step ordering matters enormously. The skill said 'search Spotify first', but the agent called all tools simultaneously until I forced sequential calls. Explicit ordering isn't enough, and you must enforce it at the framework level.

Takeaways

1. Skills enforce structure and honesty. Even when APIs fail, the agent behaved more predictably with the skill than without.
2. The hardest part may not be writing the skill, but knowing which APIs are reliable enough to build workflows on.
3. Step ordering should be enforced at the framework level, not just described in prose.
4. Music theory ground truth is ambiguous, where C# major, Db major, and C minor can all describe the same song. This is a challenge for automated analysis.